Computer Networks

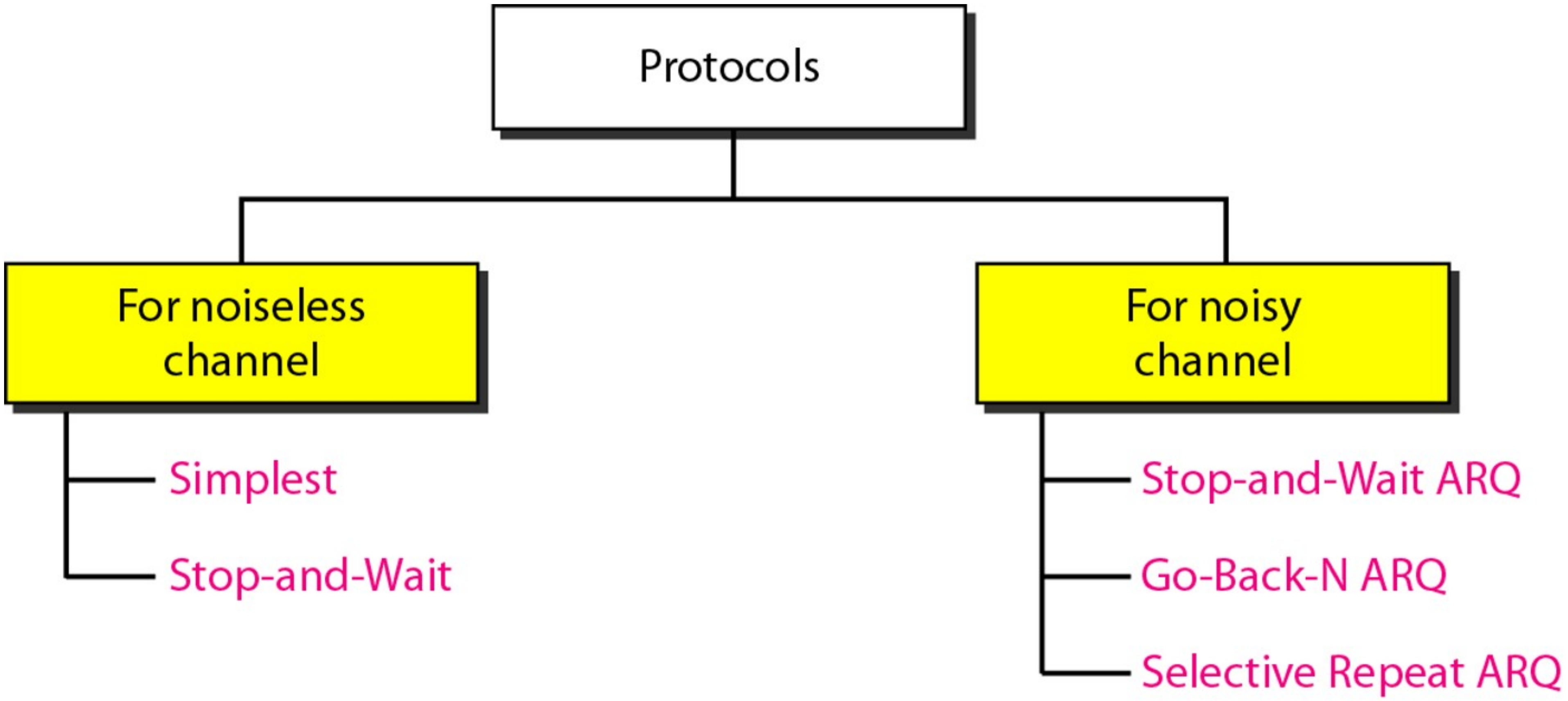
Lecture 11: Data Link Layer

Dr. Tauheed Ahmed

Content

- **Noiseless channel protocol**

Data Link Control Protocol



Simplest Protocol

It is called "simplest" because it assumes an ideal communication environment with no errors and no need for control mechanisms.

- Data flows **only in one direction**: **Sender → Receiver**
- The receiver never sends feedback/ acknowledgments.

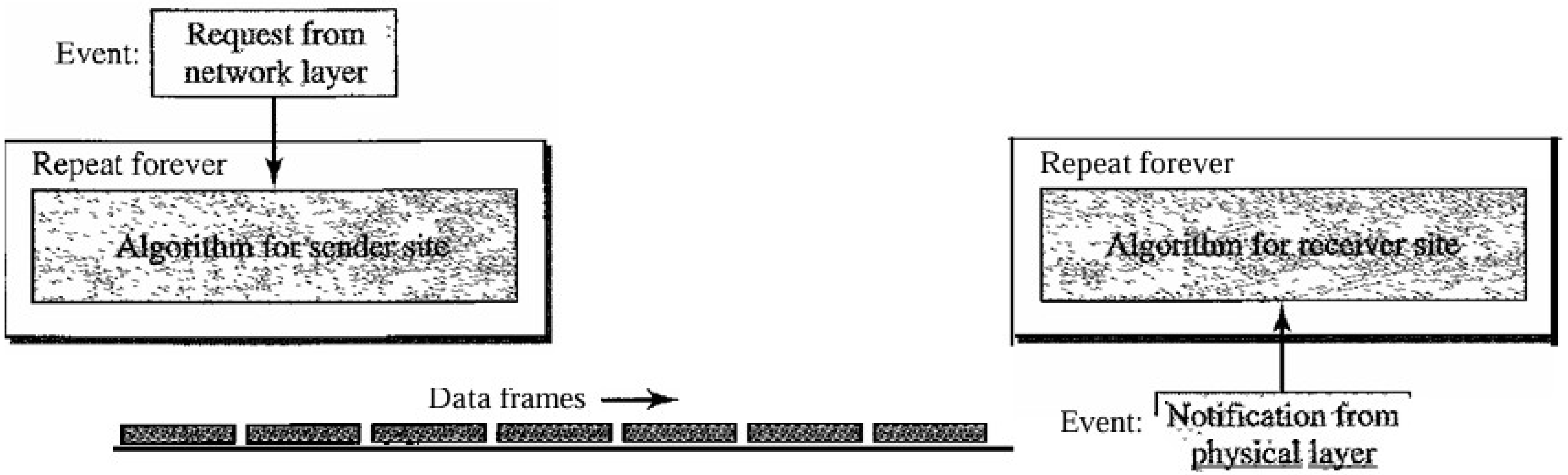
No Flow Control

- The sender can transmit frames continuously without checking whether the receiver is ready.
- It assumes the receiver is fast enough to handle all incoming frames.

No Error Control

- There is no detection or correction of errors.
- Corrupted frames are not identified or retransmitted.

Simplest Protocol




```
while (true) {                    // Repeat forever
   WaitForEvent();               // Sleep until an event occurs
   if (Event(RequestToSend)) {  // Packet to send
       GetData();
      MakeFrame();
      SendFrame();    } }      // Send the frame
```

```
while (true)     {       // Repeat forever
   WaitForEvent();    // Sleep until an event occurs
   if (Event(ArrivalNotification)) {  // Data frame
                                         arrived
      ReceiveFrame();
      ExtractData();
      DeliverData();    }} // Deliver data
```

Stop and Wait Protocol

It is called stop and wait because the sender sends one frame and then waits for confirmation before sending the next.

Direction of Communication

- **Data frames:** Unidirectional (Sender → Receiver)
- **ACK frames:** Opposite direction (Receiver → Sender)

Flow Control

- Unlike the Simplest Protocol, flow control is introduced here.
- Sender cannot transmit continuously.
- Sender transmission speed is controlled by receiver feedback.

Stop and Wait Protocol

It is called stop and wait because the sender sends one frame and then waits for confirmation before sending the next.

Approach

- **Step 1:** Sender sends one data frame.
- **Step 2:** Sender stops and waits.
- **Step 3:** Receiver processes the frame and sends an acknowledgment (ACK).
- **Step 4:** After receiving ACK, sender sends the next frame.

Stop and Wait Protocol

Sender-site algorithm for Stop-and-Wait Protocol.

```
while (true)                                  IIRepeat forever
canSend = true                                IIAllow the first frame to go
{
  WaitForEvent()i                             II Sleep until an event occurs
  if(Event(RequestToSend) AND canSend}
  {
      GetData()i
      MakeFrame();
      SendFrame()i                            I/Send the data frame
      canSend = false;                        I/cannot send until ACK arrives
  }
  WaitForEvent()i                             II Sleep until an event occurs
  if(Event(ArrivalNotification)              /1 An ACK has arrived
   {
      ReceiveFrame();                         I/Receive the ACK frame
      canSend = true;
   }
}
```

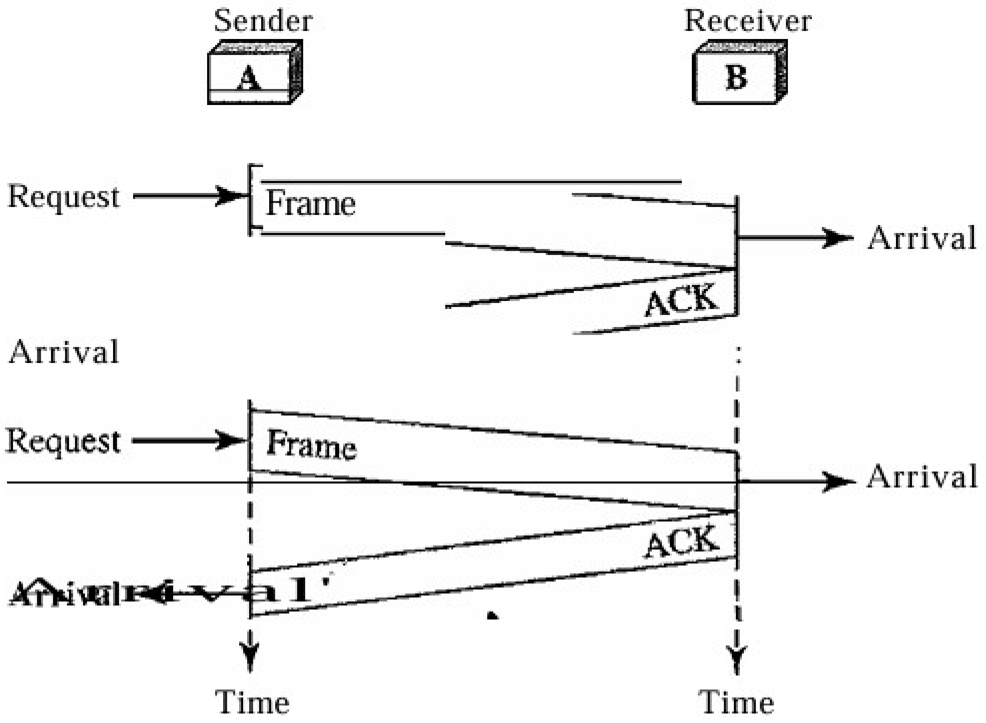
Stop and Wait Protocol

Receiver-site algorithm for Stop-and-Wait Protocol.

```
while (true)                          IIRepeat forever
{
  WaitForEvent();                     II Sleep until an event occurs
  if(Event(ArrivalNotification)}      IIData frame arrives
  {
      ReceiveFrame(};
      ExtractData(}i
      Deliver(data};                  /IDeliver data to network layex
      SendFrame();                    IISend an ACK frame
  }
}
```

Stop and Wait Protocol

Flow Diagram



Question

A Stop-and-Wait protocol is used over a noiseless channel.

Data rate = 1 Mbps
Frame size = 1000 bits
Propagation delay = 5 ms (one way)
ACK size negligible

Find the channel utilization.

Stop-and-Wait Automatic Repeat Request

Stop-and-Wait Automatic Repeat Request (Stop-and Wait ARQ), adds a simple error control mechanism to the Stop-and-Wait Protocol.

The sender sends one frame at a time and waits for an acknowledgment (ACK). If an error occurs, the frame is retransmitted automatically.

Why Error Control Is Needed?

In real networks:

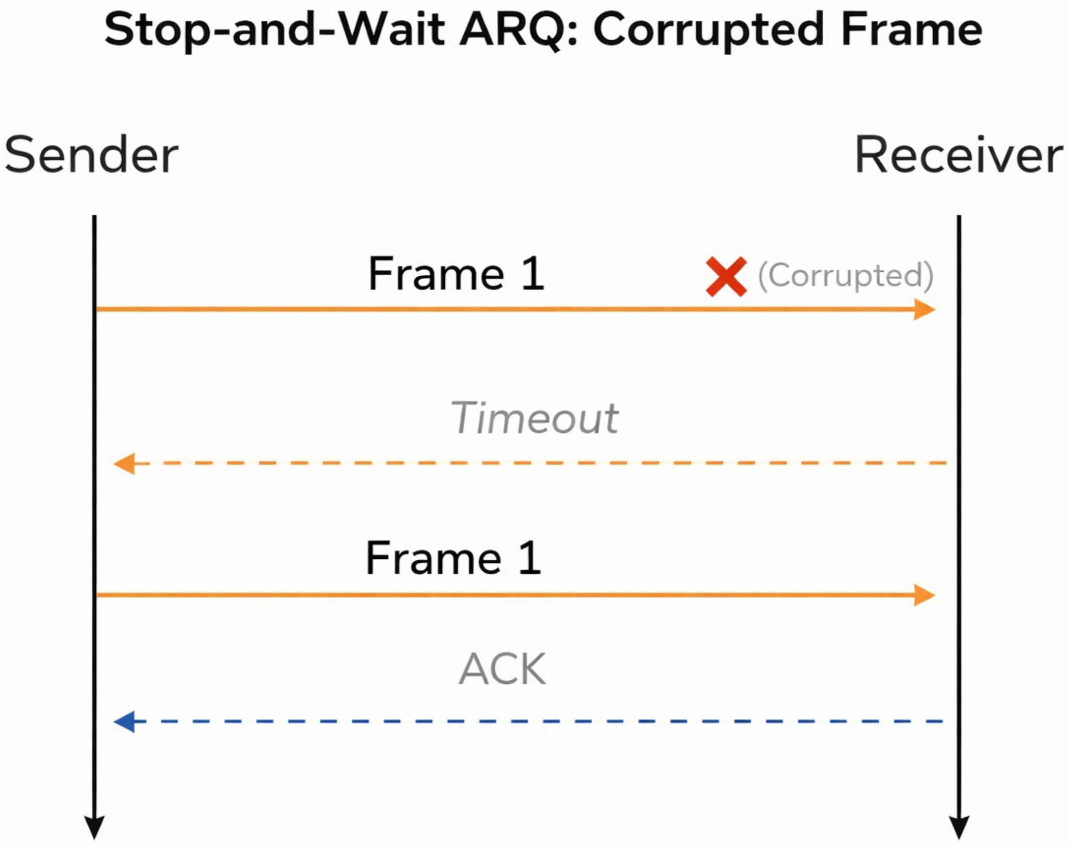
- Frames can be corrupted
- Frames can be lost
- Frames can be duplicated
- Frames can arrive out of order

So, we must detect errors and retransmit frames.

Stop-and-Wait Automatic Repeat Request

Handling Corrupted Frames

- Sender sends frame.
- Receiver checks it.
- If corrupted → receiver does nothing.
- Sender waits but receives no ACK → assumes error → retransmits.



Stop-and-Wait Automatic Repeat Request

Problem of Lost Frames

- Receiver never sees the frame.
- Sender does not know whether:
  - Frame was lost
  - ACK was lost
  - Receiver is slow

Solution

Each frame is given a **sequence number** (0, 1, 2, 3…).

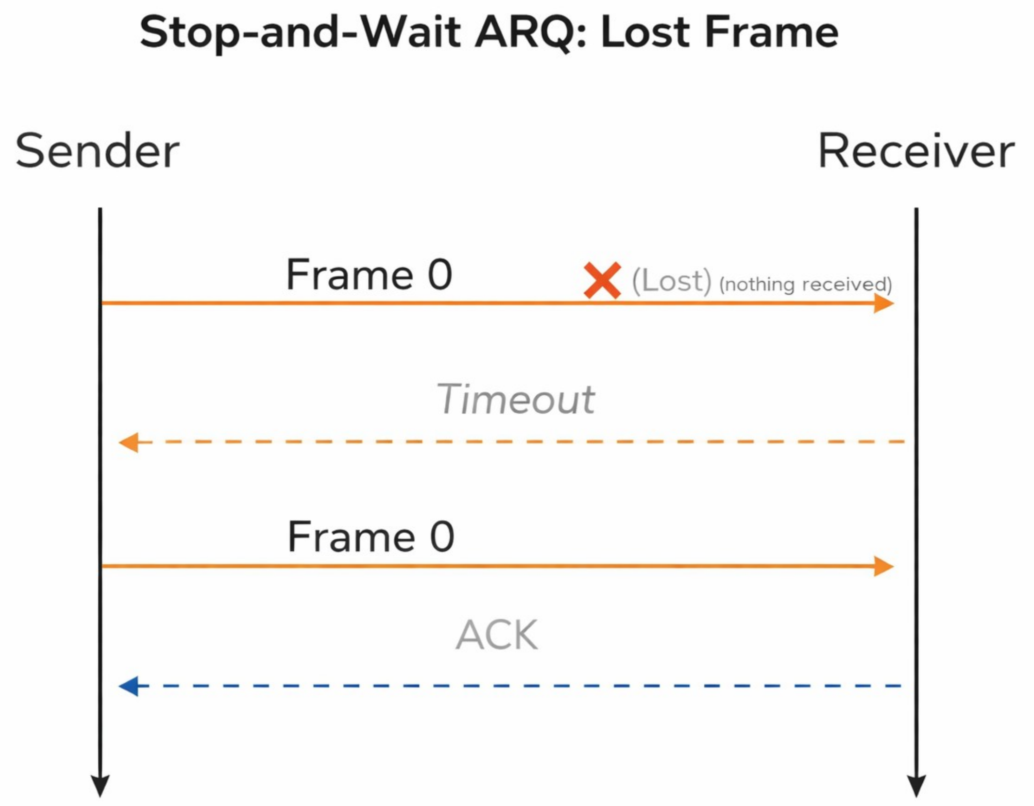
This helps to:

- Detect **duplicate frames**
- Detect **lost frames**
- Maintain **correct order**

Stop-and-Wait Automatic Repeat Request

Handling Lost Frames

- Sends frame.
- Keeps a copy of the frame.
- Starts a timer.
- If ACK arrives:
  - Stop timer.
  - Send next frame.
  - If timer expires:
  - Assume frame or ACK lost.
  - Resend the frame.



Error correction in ***Stop-and-Wait ARQ*** is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

Stop-and-Wait Automatic Repeat Request

Sequence Number

A field is added to the data frame to hold the sequence number of that frame.

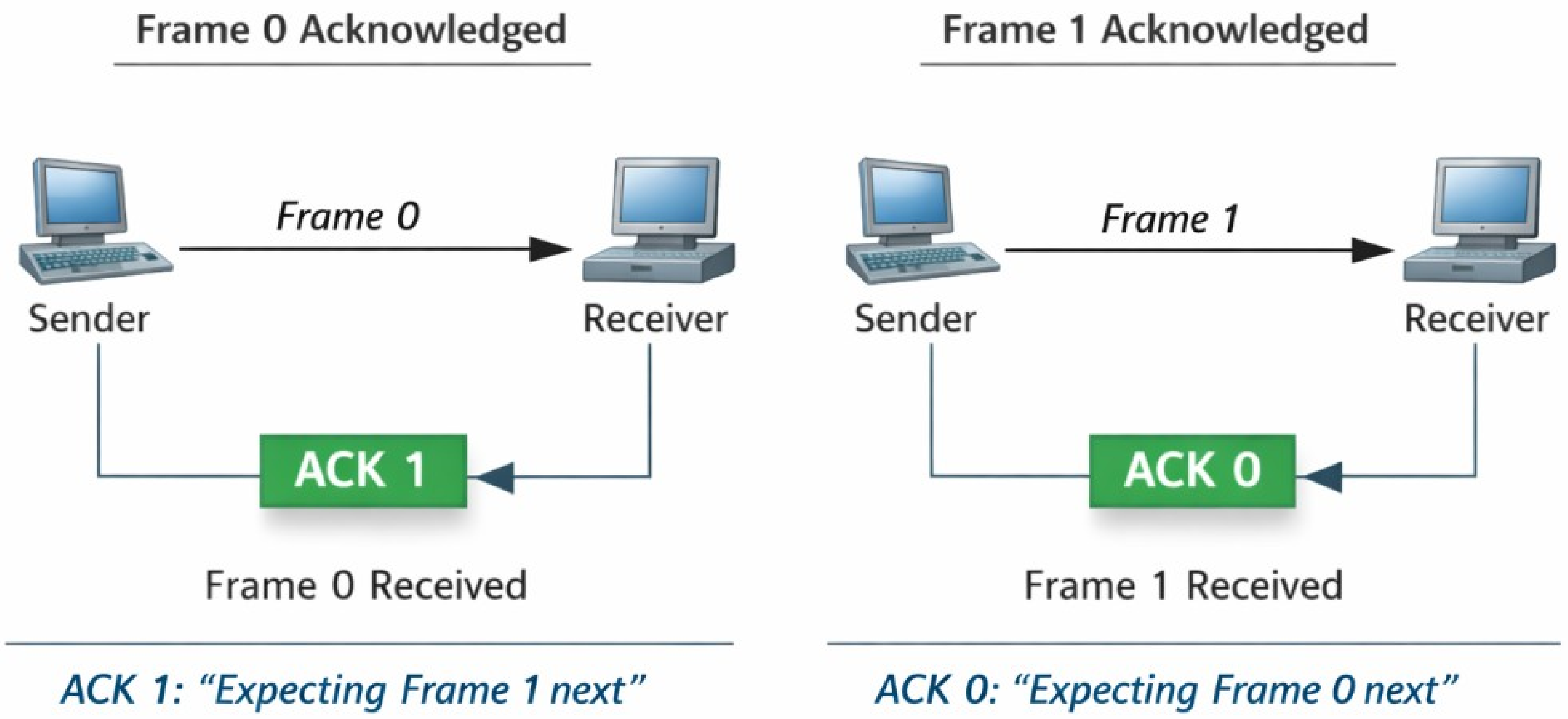
- Sequence numbers are used to distinguish new frames from retransmissions.
- In Stop-and-Wait ARQ, only two sequence numbers are needed: x and x+1.
- This is because the receiver only needs to know whether a received frame is new or duplicate.
- Therefore, sequence numbers can alternate as 0, 1, 0, 1, 0, 1 ....

Hence, a *1-bit* sequence number is sufficient for Stop-and-Wait protocol.

Stop-and-Wait Automatic Repeat Request

Acknowledgment Numbers

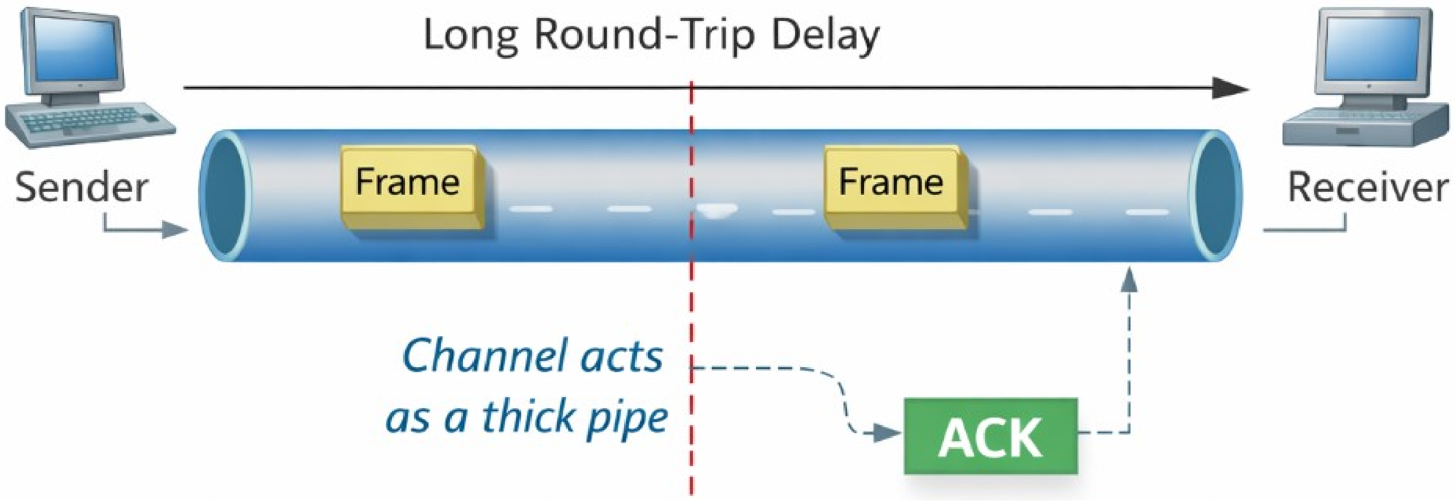
The acknowledgment number always announces in modul0-2 arithmetic the sequence number ofthe next frame expected



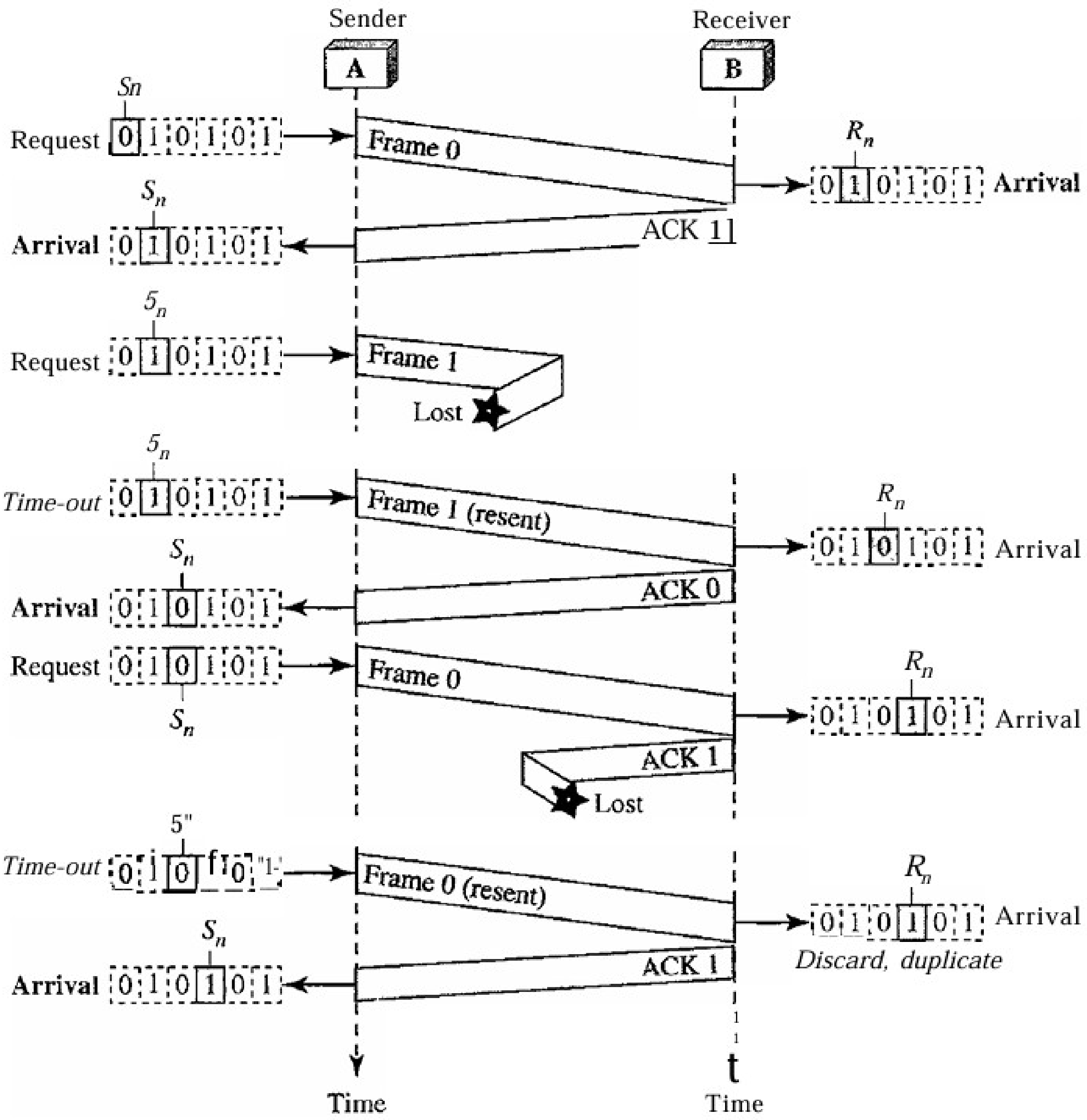
Stop-and-Wait Automatic Repeat Request

Efficiency

- Stop-and-Wait ARQ is inefficient for thick and long channels.
  - Thick channel → high bandwidth.
  - Long channel → large round-trip delay (RTT).
- Bandwidth–Delay Product (BDP) = Bandwidth × Delay.



- Channel is like a pipe; BDP is the pipe volume in bits.
- If the pipe is not kept full, bandwidth is wasted.
- BDP shows how many bits can be in transit while waiting for ACK.



Stop-and-Wait Automatic Repeat Request

Flow Diagram

Question

Assume that, in a Stop-and-Wait ARQ system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make around trip. What is the bandwidth-delay product? If the system data frames are 1000 bits in length, what is the utilization percentage of the link?